

JomCode Coding Bootcamp

Strings, input and conversion

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Normalize text without destroying meaningful internal spaces.
- 02 Use indexing, slicing, methods and f-strings.
- 03 Convert known valid terminal input to numbers.

Plain meaning → technical view → use

PLAIN MEANING

Information typed by a user starts as text.

TECHNICAL VIEW

String methods, slicing, f-strings and explicit conversion transform str values.

WHEN YOU USE IT

Clean workshop titles and turn validated capacity input into an integer.

TIP

Input is text until you convert it; test a blank and an invalid value.

Worked example 1: Index and slice



chapter_04_strings_input_and_conversion.py

```
1 title = "Python"
2 print(title[0], title[-1])
3 print(title[:3])
4 print(title[3:])
5 print(title[20:])
```

PREDICT Which operation would fail on an empty string?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
P n
Pyt
hon
[empty line]
```

WHY THIS HAPPENS

Indexing needs an existing position. Empty slices remain valid strings.

Exercise 1: Title preview

BUILD

Create a five-character title preview using a slice. It must work for short and empty titles.

CHECK

```
"Python" => "Pytho". "Go" => "Go". "" => "".
```

IF STUCK

An exclusive slice stop allows a short input.

Worked example 2: Careful normalization



chapter_04_strings_input_and_conversion.py

```
1 name = "  Ada  Lovelace  "  
2 email = "  ADA@EXAMPLE.COM  "  
3 print(name.strip())  
4 print(email.strip().lower())
```

PREDICT Should every internal double space be collapsed?

Example 2: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
Ada Lovelace  
ada@example.com
```

WHY THIS HAPPENS

Outer whitespace is accidental here. Internal text belongs to the name and stays unchanged.

Exercise 2: Email normalization

BUILD

Trim and lowercase an email. Preserve internal whitespace in a person's name.

CHECK

" ADA@EXAMPLE.COM " => "ada@example.com". " Ada L. " => "Ada L."

IF STUCK

strip and lower solve different problems.

Worked example 3: Known valid input



chapter_04_strings_input_and_conversion.py

```
1 capacity_text = input("Capacity: ")
2 registered_text = input("Registered: ")
3 capacity = int(capacity_text)
4 registered = int(registered_text)
5 print(f"Remaining: {capacity - registered}")
```

PREDICT What happens if Capacity is twenty?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
With stdin 20 then 8: Capacity: Registered: Remaining: 12
```

WHY THIS HAPPENS

int raises ValueError. Later validation handles that boundary instead of silently accepting nonsense.

Exercise 3: Fee message

BUILD

Convert the valid text "12.5" to float and create the message Fee: 12.50.

CHECK

12.5 => two decimal places. 0 => Fee: 0.00. Conversion is explicit.

IF STUCK

Keep conversion separate from display formatting.

Checkpoint

PROVE IT

Normalize workshop-related text, preserve names, and explain input conversion failures.